

DWA TORY — specyfikacja techniczna

Aplikacja do śledzenia celów dla dwóch osób (Grzesiek i Wiola), które realizują własne cele osobno, ale wzajemnie się motywują.

Dokument powstał z pełnej sesji projektowej. Zawiera nie tylko **co** zbudować, ale też **dlaczego tak, a nie inaczej** — te uzasadnienia są istotne, bo kilka pozornie oczywistych rozwiązań zostało świadomie odrzuconych i nie należy ich przywracać.

1. Czym ta aplikacja JEST, a czym NIE JEST

Jest

Narzędziem do świadomego gospodarowania czasem i konsekwentnej realizacji celów, gdzie dwie osoby widzą nawzajem swoje postępy i mogą się dopingować.

Nie jest — i to były świadome decyzje, nie przeoczenia

Odrzucone	Dlaczego
Rywalizacja, punkty, ranking	Życie domowe to nie zawody. Punkty za zadania sprawiają, że gorszy tydzień (choroba, spiętrzenie w pracy) wygląda jak porażka, choć nią nie jest.
Nagrody za punkty	Wyjątek: uroczyste pop-upy przy kamieniu milowym. Nic poza tym — żadnego sklepu z nagrodami, żadnej waluty.
Punktowanie opieki nad dzieckiem	Zamiana czasu z córką w „zadanie za punkty” jest emocjonalnie ryzykowna i sprowadza rodzicielstwo do poziomu wynoszenia śmieci.
Podział obowiązków domowych	Pierwotny pomysł, porzucony. Obowiązki to logistyka, którą można omówić na głos — apka nie dodaje tam wartości.
Time-tracking, logowanie „ile zajęło”	Twarda zasada: żadne pole nigdy nie prosi o rekonstrukcję dnia . Aпка do odzyskiwania czasu nie może sama go pożerać.
Deklarowanie	

„ile mam wolnych godzin”	Testowane i odrzucone dwukrotnie (najpierw jako „sezony”, potem jako liczba godzin). Nikt nie umie tego wiarygodnie przewidzieć. Zastąpione obserwacją realnych przesunięć.
Wspólny wykres godzin obojga	Prowadziłby do „ty masz 2h, ja 1h”, nawet gdy różnica jest w pełni uzasadniona. Zamiast tego: każde widzi swoje dane, kontekst tłumaczy różnice.

Zasada nadrzędna

Próg wejścia: ~10 sekund dziennie. Domyślny widok musi być chudy. Wszystko opcjonalne chowa się za świadomym kliknięciem.

2. Skąd nazwa (i dlaczego to ważne dla designu)

„Dwa Tory” działa na dwóch poziomach, i oba są realnie zakodowane w interfejsie:

1. **Dwa typy celów** — cel z terminem i kamieniami milowymi (kolor koralowy) kontra nawyk cykliczny (kolor zielonkawy). To nie metafora — to różnica w tym, jak się z czymś pracuje.
2. **Dwie osoby** — osobno w Dzienniku i Celach, zbiegające się w Kalendarzu i we „Wspólnej serii”.

Warstwa wizualna konsekwentnie to odzwierciedla: pasek boczny dzielony na pół w widoku wspólnym, gradient łączący oba kolory, tor kropek jako linia postępu.

3. Stos technologiczny — DECYZJE OSTATECZNE

Platforma: PWA (web), NIE natywna

Rozważono trzy ścieżki, wybrano PWA:

	PWA 	Capacitor + Dev Program	Capacitor + AltStore
Koszt	0 zł	~350 zł/rok	0 zł
Instalacja dla Wioli	link + „Dodaj do ekranu”	TestFlight	własna konfiguracja AltStore

Utrzymanie	zero	re-upload co 90 dni	odświeżanie co 7 dni, obustronnie
Push	słabszy (Safari 16.4+)	pełny APNs	brak
CloudKit	✗	✓	✓

Uzasadnienie: zero kosztów i zero obsługi po obu stronach przeważały nad mocniejszym pushem. Instalacja to wysłanie linku.

Co z tego wynika

- **Frontend:** React + Tailwind (dokładnie jak makiety), `manifest.json` + service worker dla instalowalności i trybu offline.
- **Backend:** Supabase (rekomendacja) albo Firebase — darmowe plany z ogromnym zapasem przy 2 użytkownikach.
- **Auth:** własny system kont (Supabase Auth / Firebase Auth). CloudKit odpada — jest wyłącznie natywny.
- **Storage lokalny:** IndexedDB / localStorage dla trybu offline i natychmiastowego startu.

⚠ KRYTYCZNE: `window.storage` w makietach NIE jest prawdziwym API

Plik `dwa-tory-finalna.jsx` używa `window.storage` — to mechanizm istniejący **wyłącznie wewnątrz Claude.ai**, dodany tylko po to, żeby makietę pamiętała dane między odświeżeniami podczas projektowania. **Poza Claude.ai to API nie istnieje i kod się nie uruchomi.** Zastąpić: IndexedDB lokalnie + Supabase/Firebase zdalnie.

Wizualnie zero kompromisu

Cały styl (gradienty, `backdrop-filter`, animacje `@keyframes`) to zwykły CSS — przechodzi z makiet 1:1 bez tłumaczenia. To był argument rozstrzygający porównanie technologiczne (React Native wymagałby przepisania każdej animacji i dołożenia bibliotek do gradientów i blura).

4. Model danych

Osoba (`person`)

```
id, name, initials, color, photo: { src } // src = przycięty JPEG data URL
```

```
streak, longestStreak, cheers
```

Cel (goal)

```
id                // crypto.randomUUID()
title
type              // "termin" | "cykliczny" → determinuje kolor (TYPE_COLOR)
character         // "habit" | "termin" | "cyclicalContent" (z kreatora)
reason           // opcjonalne "dlaczego to robię"
anchor           // opcjonalne "po czym to zrobię" (habit stacking)
minimalVersion    // opcjonalna wersja awaryjna
start            // data startu – punkt "Start" w Twojej podróży
cadenceLabel     // czytelny opis: "codziennie", "Pn, Śr o 18:00", "3× w tygod
cadenceSlots     // ["Dziś", "Jutro"] dla wzorców dnia-konkretnego
                 // ["Ten tydzień", "Przyszły tydzień"] tylko dla "X razy w tyg
targetValue,     // tylko dla type="termin": co mierzymy
targetUnit
completedSessions // tylko dla type="termin": licznik napędzający kamienie milo
milestones[]     // { id, label, date, threshold? }
instance         // { curr: {status, note}, next: {status, double} }
rescheduleCount  // licznik przesunięć – napędza ostrzeżenie o przeciążeniu
visibleToPartner // bool
syncToPhoneCalendar // bool
```

Uwaga: `hoursPerWeek` zostało usunięte — relik po odrzuconym systemie capacity. Nie przywracać.

Stany instancji zadania

```
plan → done | moved | skipped
```

Kluczowe: `moved` i `skipped` **nie są porażką**. Mają neutralne kolory (bursztynowy / szary), nie czerwony. Przesunięcie nie zeruje serii.

Kamienie milowe — dwa tryby

1. **Liczone automatycznie** (`completedSessions >= threshold`) — dla celów typu „100 dni treningu”. Funkcja `milestonesFor(goal)` przelicza status na bieżąco; **nie przechowywać `done` w danych** dla takich celów.
2. **Ręczne** — dla celów z treścią (rozdziały książki, moduły kursu), gdzie postępu nie da się wywnioskować z licznika sesji.

Progi muszą rosnąć monotonicznie, bez duplikatów, ostatni zawsze równy `targetValue`.

5. Ekrany

5.0 Splash

„DWA TORY” na środku, pojawia się i znika (~1,7s). Pokazuje się przy **każdym uruchomieniu apki**, ale nie przy przełączeniu karty/powrocie z innej aplikacji (stan tylko w pamięci komponentu, nie w storage).

5.1 Onboarding (plik: *onboarding.jsx*)

Przepływ: powitanie → imię + zdjęcie → połączenie z partnerką → pytanie o „czas dla siebie” → wstęp do samouczka → 4 karty samouczka → gotowe.

- **Zdjęcie profilowe:** aparat albo galeria, potem realne kadrowanie (przeciąganie + zoom), zapis przez `<canvas>` jako skompresowany JPEG.
- **Połączenie:** „Wygeneruj kod” LUB pole na kod od partnerki — **zawsze obie opcje naraz**, bo nie wiadomo z góry kto inicjuje. Opcja „zrobię to później” nie blokuje onboarding.
- **„Czas dla siebie”:** pytanie tutaj decyduje, czy funkcja w ogóle jest widoczna w apce. Domyślnie wyłączona.
- **Samouczek:** można pominąć TYLKO na ekranie wstępu. Po wejściu w pierwszą kartę trzeba przejść wszystkie cztery. Uruchamialny ponownie z Profilu.
- Miniaturki w samouczku muszą wyglądać jak prawdziwe ekrany (te same kolory, kształty), nie generyczne ilustracje.

5.2 Nagłówek (stały, na wszystkich zakładkach)

Lewa: nazwa „DWA TORY”. Prawa: licznik serii (🔥 tylko na Dzienniku), miniatura profilu (→ Profil), dzwoneczek z liczbą nieodpowiedzianych powiadomień.

Zasada: cyfra na miniaturce pojawia się wyłącznie przy dzwoneczku. Awatar zawsze czysty.

5.3 Dziennik (główny)

Jedna karta zawierająca wszystkie moje cele jako równoległe tory — **nie stos osobnych kart** (to zapobiega zamienieniu się w zwykłą listę to-do przy większej liczbie celów).

Każdy tor: kropka koloru typu, tytuł, mini-tor kamieni (tylko jeśli cel ma kamienie), chipy „Dziś/Jutro”, duży przycisk „**Zrobione**” i mały, cichy link „**przesuń**”.

Hierarchia jest celowa: „Zrobione” jest wyraźnie dominujący (~3/4 szerokości, pogrubiony,

wypełniony kolorem). „przesuń” to tekst bez obwódki. Zależy nam na realizacji, nie na odraczaniu.

- Notatka: chowa się za linkiem „+ notatka”, nigdy nie widoczna domyślnie. Do wglądu, nie wpływa na nic.
- Wszystkie tory działają identycznie — brak skrótów typu boczny checkbox (odrzucone: spójność ważniejsza niż skrót).
- Gradient tła karty nasila się wraz z długością serii.
- Gdy wszystkie dzisiejsze zadania zrobione: złota obwódka (stan trwały) + isierki (jednorazowa animacja).
- Opcjonalnie na dole: „Czas dla siebie” (1h / 2h / Wieczór) — tylko jeśli włączone w ustawieniach.

Mechanika przesuwania — kaskada

Kliknięcie „przesuń” sprawdza następny slot tego samego celu:

- Slot wolny → po prostu przesuwam.
- **Slot zajęty → wymusza świadomą decyzję** (nie cichy overwrite): „Zrób oba naraz (2x)” / „Odpuść dzisiejsze” / „Odpuść jutrzejsze”.

To jedyne miejsce, gdzie apka w ogóle wspomina o obciążeniu dnia — i robi to bez pytania o godziny.

Pętla zwrotna: przesunięcia jako sygnał przeciążenia

Przy `rescheduleCount >= 2` na karcie celu (zakładka Cele) pojawia się delikatne ostrzeżenie: „2x przesunięte — sprawdź tempo”. **To zastępuje cały odrzucony system deklarowania godzin** — zamiast prosić o prognozę, obserwuje fakty.

5.4 Cele

Tylko moje cele (edycja cudzych nie ma sensu przy osobnych kontach).

Karta celu: kropka typu, tytuł, tor kamieni albo „nawyk · codziennie”, ostrzeżenie o przesunięciach jeśli dotyczy, strzałka > sygnalizująca klikalność. Kliknięcie → edycja.

Kolor **zawsze** z `TYPE_COLOR[goal.type]`, nigdy z koloru osoby — ten sam cel musi wyglądać identycznie we wszystkich zakładkach.

5.5 Kreator / Edytor celu

Jeden komponent w dwóch trybach. Wejście: FAB (nowy) albo karta celu (edycja).

Kroki (liczba zależna od gałęzi):

1. **Nazwa + „dlaczego” + typ** (szybkie zadanie / cel do śledzenia) + charakter (nawyk bez końca / cel z targetem / cel cykliczny z treścią)
2. **Wzorzec powtarzalności** — Codziennie / Konkretny dzień tygodnia (+ godzina) / X razy w tygodniu (+ preferowana pora) / Co miesiąc (+ **wymagany** dzień 1–31) — oraz **start** (dziś / wybierz z kalendarza) i **kotwica** („po czym to zrobisz”)
3. **Wersja minimalna** (opcjonalna) + automatycznie wyliczony limit 25%
4. *(tylko cele z kamieniami)* **Mapa kamieni** — kalendarz, stuknięcie dnia dodaje etap, lista pod spodem z nazwami i usuwaniem. Dla celu z terminem dodatkowo: co mierzymy (wartość + jednostka) i licznik kamieni
5. **Podsumowanie** + widoczność dla partnerki + sync z kalendarzem telefonu

Szybkie zadanie ma skróconą ścieżkę: nazwa → kalendarz + godzina → podsumowanie.

Tryb edycji dodatkowo: kroki klikalne (nawigacja swobodna), „Zapisz zmiany”, „Usuń cel” z potwierdzeniem.

Ostrzeżenie przy porzuceniu: jeśli formularz zawiera dane i użytkownik cofa się z kroku 0 — pytanie „na pewno?”.

Uzasadnienie psychologiczne pytań

- **Kotwica (habit stacking)** — najsilniejszy potwierdzony mechanizm utrzymania nawyku. „Codziennie” bez punktu zaczepienia w dniu to sama intencja.
- **„Dlaczego”** — jedno zdanie zapisane raz przy starcie; daje treść, do której można wrócić.
- **Wersja minimalna** — ratuje serię w złych dniach, ale **oznacza dzień inaczej niż pełne wykonanie** (apka nie okłamuje użytkownika). Limit 25% dni w miesiącu, wyliczany z kadencji, żeby nie stało się to domyślnym trybem.
- **Disclaimer o pomijaniu** — użytkownik dowiaduje się, że pominięcie jest OK, PRZED pierwszą wpadką, nie po.

5.6 Kalendarz (połączony ze statystykami)

Przełącznik u góry (jak w Outlooku): **Mój / Wiola / Wspólny**. Drugi przełącznik: **Tydzień** (domyślny) / **Miesiąc**. Oba domyślne ustawiane w Profilu.

- **Widok tygodniowy:** siedem wierszy, pasek koloru po lewej (w trybie „Wspólny” dzielony na pół — góra Ty, dół Wiola), nazwa kamienia milowego wypisana wprost.
- **Widok miesięczny:** siatka dni, kolor = stan dnia, złota obwódka = kamień milowy, biała obwódka = dziś.
- **W trybie „Wspólny”:** pasek „Wspólna seria” — gradient łączący oba Wasze kolory, wyśrodkowana etykieta i liczba dni, w których **oboje** mieliście zrobione. Nie da się zdobyć w pojedynkę. To jedyny pełny gradientowy blok w apce — pozostałe elementy używają koloru oszczędniej, żeby ten pozostał wyjątkowy.
- Pod spodem: „**Do zrobienia**” i „**Zrealizowane**”, po 5 pozycji z opcją „Pokaż więcej”.
- Codzienne nawyki bez kamieni **nie pojawiają się** w Kalendarzu — to widok dużych rzeczy, Dziennik jest od codzienności.

5.7 Profil

- Awatar + imię
- **Podsumowanie:** bieżąca seria / rekord / zdobyte kamienie
- **Twoja podróż:** pionowa oś czasu dla **każdego** celu z liczonym postępem — Start → zdobyte kamienie → pulsujące „Dziś — dzień X” → przyszłe (puste kropki)
- **Ustawienia** — wszystkie pozycje rozwijają się w miejscu, jednym wzorcem (klik → obrót strzałki → rozwinięcie):
 - Powiadomienia (push, dźwięk)
 - Konto i połączenie z partnerem (status + rozłącz / wygeneruj kod + wpisz kod)
 - Zdjęcie profilowe (aparat / galeria → kadrowanie)
 - Czas dla siebie (włącz/wyłącz)
 - Domyślny widok kalendarza (Mój/Wiola/Wspólny + Tydzień/Miesiąc)
 - O aplikacji (wersja)
 - *(do dodania)* Uruchom samouczek ponownie

5.8 Powiadomienia

Panel **wysuwany od góry** (nie osobny ekran). Dzwoneczek działa jak przełącznik.

Zamykanie: przycisk „zwiń”, przeciągnięcie **uchwyty** w górę (uchwyt, nie cały panel — inaczej koliduje ze scrollem), albo zmiana zakładki.

Lista wpisów: co partnerka zrobiła + kiedy. Przy nieodpowiedzianych: cztery gotowe chipy albo własny tekst z **twardym limitem 5 słów** (licznik, blokada wysyłki po przekroczeniu). Odpowiedziane pokazują wysłaną treść z ✓.

Dlaczego limit słów: to ma być symboliczny gest docenienia, nie czat. Apka nie ma być komunikatorem.

6. Integracje

Kalendarz telefonu (opcjonalny, per cel)

- Jednokierunkowo: apka → kalendarz. Bez odczytu zwrotnego.
- Wymaga zgody użytkownika przy pierwszym użyciu.
- **Przy przesunięciu zadania: usuń stare wydarzenie po zapamiętanym `eventId`, utwórz nowe.** Nigdy nie zostawiać duplikatu (stary wpis wysłałby fałszywe przypomnienie).
- Wymaga przechowywania `eventId` przy instancji.

Łączenie kont

1. Oboje mają konta (Supabase/Firebase Auth).
 2. „Wygeneruj kod” → serwer tworzy tymczasowy wpis (kod → ID konta), ważny ~15 min.
 3. Druga osoba wpisuje kod → serwer trwale łączy oba ID w rekord „pary”.
 4. Zmiany trafiają na serwer, nie tylko lokalnie.
 5. Druga osoba dowiaduje się przez realtime (Supabase Realtime / Firebase), nie przez ręczne odświeżanie.
-

7. Design system

bg	#12211D	tło aplikacji
surface	#1B322B	karty
surface2	#213C33	zagnieżdżone elementy
line	#2C4A40	obramowania
gold	#E3A542	akcent, seria, powiadomienia
text	#F3EFE4	tekst główny
muted	#9FB3AC	tekst drugorzędny

```
skipped    #5A6B65    dzień pominięty
over       #D9604E    ostrzeżenie / usuwanie

TYPE_COLOR.termin    #E8724F    cel z terminem
TYPE_COLOR.cykliczny  #8AAE9E    nawyk / cel cykliczny
```

Osoby: Grzesiek #E8724F, Wiola #5FA8AE . ⚠️ Uwaga: kolor osoby i kolor typu celu to dwa różne wymiary, które przypadkiem mają zbliżone wartości. W repo muszą być osobnymi, jasno nazwanymi stałymi w jednym `theme.ts` .

Typografia: Bebas Neue (nagłówki/liczby), Fraunces (podtytuły), Inter (tekst).

Animacje:

- `flame-burst` — płomień przy odhaczeniu (0,6s)
- `milestone-overlay` — pełnoekranowa nakładka kamienia z blurem tła (3,4s)
- `marker-pulse` — pulsowanie bieżącej pozycji na osi
- `splash` — ekran startowy (1,7s)

Do dopracowania: różne animacje w zależności od wagi kamienia (100% powinno być wyraźnie mocniejsze niż 25%).

⚠️ Luka do domknięcia: widoczność celu dla partnerki

W makiecie `visibleToPartner` istnieje tylko jako pole w formularzu — **nie jest zapisywane na obiekcie celu i nigdzie nie filtruje danych**. Kalendarz w widoku "Wspólny" (`milestonesOf`) pokazuje wszystkie cele danej osoby bez wyjątku. To realny wyciek prywatności, nie tylko brakująca funkcja — w repo musi być egzekwowane: cel z `visibleToPartner: false` nie może pojawić się w żadnym widoku, do którego ma dostęp druga osoba (Kalendarz "Wspólny"/"Wiola", "Wspólna seria", ewentualne przyszłe funkcje).

⚠️ Luka do domknięcia: odzyskiwanie dostępu przy nowym telefonie

Model logowania jest oparty o konto (Supabase/Firebase Auth), nie o urządzenie, więc technicznie zalogowanie się na nowym telefonie powinno wystarczyć — ale nigdzie nie zaprojektowaliśmy tego jako świadomy przepływ (reset hasła, ponowne parowanie po utracie sesji, co się dzieje ze zdjęciem/danymi lokalnymi z poprzedniego urządzenia). Do zaprojektowania przed pierwszym realnym wdrożeniem, nie po fakcie zgubienia telefonu.

! Luka do domknięcia: eksport / backup danych

Po kilku miesiącach realizacji celu "Twoja podróż" gromadzi dane, na których komuś zależy. Nie ma żadnego mechanizmu "wyeksportuj to, co zrobiłem" — ani na wypadek zmiany usługi backendowej, ani jako zwykła kopia zapasowa. Minimalna wersja: eksport własnych danych do JSON/CSV z poziomu Profilu.

8. Wymogi jakościowe

- **Cele dotykowe** min. ~44×44px (dopełnienie + ujemny margines, żeby wizualnie nic się nie przesuwalo).
 - **Jeden moduł motywu** — `theme.ts` + `calendarUtils.ts` importowane wszędzie. Makiety mają zduplikowane definicje w trzech plikach; w repo to musi być jedno źródło.
 - **ID:** `crypto.randomUUID()` , nie `Date.now()` .
 - **Zdjęcia:** kadrowanie przez `<canvas>` + kompresja przed zapisem. Nigdy nie zapisywać surowego pliku z aparatu.
 - **Tryb offline:** apka musi działać i pokazywać dane bez sieci, synchronizować po powrocie połączenia.
-

9. Pliki makiet

Plik	Zawartość	Status
dwa-tory-finalna.jsx	Wszystkie 4 zakładki + kreator + powiadomienia + splash	źródło prawdy
onboarding.jsx	Onboarding + samouczek	aktualny
nowy-cel-kreator.jsx	Standalone kreator	referencja historyczna — logika została wchłonięta do pliku głównego
porownanie-technologiczne.jsx	Porównanie renderowania w 3 stosach	uzasadnienie decyzji
notatki-	Notatki cząstkowe	wchłonięte do tego dokumentu

Makiety pokazują **wygląd i zachowanie**, nie architekturę. Dane są zamockowane, `window.storage` jest tymczasowy, wszystko działa lokalnie w jednym komponencie.

10. Kolejność budowy (sugerowana)

1. **Fundament** — projekt PWA (manifest, service worker), `theme.ts`, routing zakładek, splash
2. **Dane lokalne** — model, IndexedDB, tryb offline
3. **Dziennik** — tory, odhaczanie, kaskada przesuwania (rdzeń codziennego użycia)
4. **Cele + kreator/edytor** — pełne drzewo decyzyjne
5. **Kalendarz** — oba widoki, kamienie milowe
6. **Profil** — statystyki, Twoja podróż, ustawienia
7. **Backend** — auth, łączenie kont, synchronizacja
8. **Powiadomienia** — panel w apce, potem push
9. **Onboarding + samouczek** — na końcu, gdy jest już co pokazywać
10. **Integracja z kalendarzem telefonu** — opcjonalna wisienka